

AI Asystent regulaminów

Aplikacja pełniąc rolę asystenta AI odpowiadającego na pytania dotyczące regulaminów i zarządzeń IH PAN.

Główne elementy:

- app.py - aplikacja Flask, routing, logowanie, API.
- rag.py - lokalna baza wiedzy z plików Markdown i wyszukiwarka fragmentów.
- llm.py - budowanie kontekstu i wywołanie Gemini.
- static/app.js - obsługa chatu w przeglądarce.
- templates/index.html - główny widok aplikacji.
- md/ - katalog źródłowych dokumentów Markdown, z których budowany jest indeks.
- wiki/ - automatycznie generowana, pomocnicza mapa tematyczna dokumentów z md/.

Aplikacja nie używa wektorowej bazy danych ani embeddingów. Wyszukiwanie jest lokalne, słownikowo-tokenowe: kod dzieli dokumenty na fragmenty, tokenizuje ich treść, a potem liczy dopasowanie słów z pytania do słów z fragmentów.

Start Aplikacji Przy imporcie app.py tworzona jest aplikacja Flask:

```
app = Flask(__name__)
app.secret_key = SECRET_KEY
```

Następnie ustawiany jest `ProxyFix`, żeby aplikacja poprawnie działała za proxy, np. z gunicornem/nginxem:

```
app.wsgi_app = ProxyFix(...)
```

Najważniejsze: od razu tworzony jest globalny obiekt bazy wiedzy:

```
kb = KnowledgeBase(MD_DIR)
```

To dzieje się w app.py. Konstruktor `KnowledgeBase` w rag.py natychmiast wywołuje `reload()`.

`reload()`:

1. Czyści listę dokumentów i fragmentów.
2. Przechodzi po wszystkich plikach *.md w katalogu MD_DIR.
3. Czyta każdy plik.
4. Dzieli dokument na fragmenty przez `split_document()`.
5. Tworzy tokeny dla każdego fragmentu.

To oznacza, że indeks jest budowany w pamięci przy starcie aplikacji, a nie przy każdym pytaniu.

Konfiguracja Konfiguracja jest w config.py.

Domyślnie:

- dokumenty są czytane z `md/`,
- pomocnicza wiki jest zapisywana w `wiki/`,
- tytuł aplikacji to **Asystent regulaminów i zarządzeń Instytutu Historii PAN**,
- model Gemini to `gemini-3-flash-preview`,
- limit wyszukiwanych fragmentów to `SEARCH_LIMIT = 8`,
- maksymalna długość kontekstu dla modelu to `MAX_CONTEXT_CHARS = 18000`.

Plik `.env`, jeśli istnieje, jest ładowany ręcznie przez `load_env_file()`. Klucz do Gemini jest brany z `GOOGLE_API_KEY` albo `GEMINI_API_KEY`.

Rola Katalogu wiki/ Katalog `wiki/` nie jest właściwym źródłem odpowiedzi chatu i nie jest czytany przez endpoint `/api/chat`. Właściwa baza wiedzy aplikacji powstaje z plików Markdown w `md/`, bo `KnowledgeBase(MD_DIR)` iteruje po `MD_DIR` i z tych dokumentów buduje listę dokumentów oraz fragmentów.

`wiki/` jest warstwą pomocniczą dla człowieka: zawiera automatycznie wygenerowaną mapę tematyczną dokumentów źródłowych. Tworzy ją `build_wiki.py`, korzystając z tej samej klasy `KnowledgeBase`, która indeksuje `md/`. Skrypt definiuje zestaw tematów, np. `regulamin-pracy`, `zfss`, `zamowienia-publiczne`, `finanse-i-rachunkowosc`, a potem dla każdego tematu wykonuje wyszukiwania po dobranych słowach kluczowych i zapisuje listę najbardziej powiązanych dokumentów.

Najważniejsze pliki w `wiki/`:

- `wiki/index.md` - indeks tematów oraz statystyki liczby dokumentów i fragmentów,
- `wiki/<temat>.md` - lista dokumentów z `md/` najbardziej pasujących do danego tematu, z datą, ścieżką, tytułem i oznaczeniem zmian/aneksów/uchyleń.

Sam plik `wiki/index.md` mówi wprost, że jest to robocza warstwa orientacyjna, a źródłem prawdy pozostają pliki w `md/`. Do przebudowania tej warstwy służy:

```
python manage_index.py build-wiki
```

albo równoważnie:

```
python manage_index.py refresh
```

Oba polecenia wywołują `build_wiki()` i po przebudowaniu pokazują statystyki indeksu. Zmienna środowiskowa `REGULAMINY_WIKI_DIR` pozwala wskazać inny katalog dla tej pomocniczej wiki.

Logowanie Każde żądanie przechodzi przez `require_login()` w `app.py`.

Jeśli użytkownik nie jest zalogowany:

- dla stron HTML dostaje przekierowanie na `/login`,
- dla endpointów `/api/...` dostaje JSON:

```
{"error": "Wymagane logowanie."}
```

Logowanie jest bardzo proste: porównuje login i hasło z konfiguracji przez `hmac.compare_digest()` w `app.py`. Po sukcesie ustawia w sesji:

```
session["authenticated"] = True
```

Ekran Główny Po wejściu na `/` działa endpoint `index()` w `app.py`.

Renderuje `templates/index.html`, przekazując:

```
title=APP_TITLE
stats=kb.stats()
```

Na stronie jest:

- nagłówek z tytułem,
- liczba dokumentów,
- ostrzeżenie, że AI może się mylić,
- obszar wiadomości `#messages`,
- formularz `#chat-form`,
- textarea `#message`,
- modal do podglądu dokumentów źródłowych.

Frontend ładuje `static/app.js`.

Proces Pytania Najważniejsza ścieżka zaczyna się w `static/app.js`.

Frontend nasłuchuje wysłania formularza:

```
form.addEventListener("submit", async (event) => {
```

Po wysłaniu:

1. Blokuje domyślne przeładowanie strony przez `event.preventDefault()`.
2. Czyta tekst z pola `textarea`.
3. Usuwa białe znaki z początku i końca przez `.trim()`.
4. Jeśli pytanie jest puste, nic nie robi.
5. Czyści `textarea`.
6. Dodaje wiadomość użytkownika do chatu.
7. Dodaje tymczasową wiadomość asystenta: Szukam w dokumentach....

Ten fragment jest tutaj: `static/app.js`.

Wiadomość użytkownika jest dodawana przez `addMessage("user", message)` z `static/app.js`. Funkcja tworzy element:

```
<article class="message user">
  <div class="message-content">...</div>
</article>
```

i dopina go do `#messages`.

Potem frontend wysyła request:

```
fetch(appUrl("/api/chat"), {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message }),
})
```

Czyli do backendu idzie JSON:

```
{
  "message": "pytanie użytkownika"
}
```

Endpoint /api/chat Backend odbiera pytanie w app.py.

Najpierw parsuje JSON:

```
payload = request.get_json(force=True)
question = (payload.get("message") or "").strip()
```

Jeśli pytanie jest puste, zwraca błąd 400:

```
{"error": "Brak pytania."}
```

Jeśli pytanie istnieje, zaczyna się właściwy RAG:

```
chunks = attach_source_ids(kb.search(question, limit=SEARCH_LIMIT))
answer = answer_with_gemini(question, chunks)
sources = cited_sources(answer, chunks)
```

To są trzy kluczowe kroki.

Krok 1: Wyszukiwanie Fragmentów kb.search() jest w rag.py.

Najpierw pytanie jest tokenizowane:

```
query_tokens = tokenize(query)
```

Tokenizacja w rag.py:

- wyszukuje słowa regexem,
- robi casefold(), czyli ujednolica wielkość liter,
- usuwa krótkie słowa o długości ≤ 2,
- usuwa stopwordy typu i, w, do, oraz, nie, się.

Przykładowo pytanie:

Czy w IH PAN ustala się plan urlopów?

zostanie zredukowane do istotniejszych tokenów, np. okolice:

pan, ustala, plan, urlopów

Następnie każdy fragment dokumentu jest oceniany.

Dla każdego chunka aplikacja sprawdza:

```
overlap = query_set & tokens
```

Czyli: jakie słowa z pytania występują też we fragmencie.

Jeśli nie ma żadnego wspólnego tokena, fragment odpada.

Jeśli jest dopasowanie, liczony jest score:

```
score = len(overlap) / math.sqrt(max(len(tokens), 1)) + phrase_bonus + recency_bonus + change_bonus
```

Score składa się z:

- liczby wspólnych tokenów, z korektą na długość fragmentu,
- `phrase_bonus`: +0.15 za każdy token pytania znaleziony w tekście,
- `recency_bonus`: +0.2, jeśli data dokumentu jest od 2024-01-01,
- `change_bonus`: +0.25, jeśli dokument wygląda na zmianę/aneks/uchylenie.

Potem wyniki są sortowane malejąco po score i dacie:

```
scored.sort(key=lambda item: (item[0], item[1].date), reverse=True)
```

Do `/api/chat` wraca maksymalnie `SEARCH_LIMIT` fragmentów, domyślnie 8.

Jak Powstają Fragmenty Fragmenty powstają wcześniej, przy starcie aplikacji, w `split_document()` w `rag.py`.

Dla każdego pliku Markdown aplikacja:

1. Wyciąga tytuł z pierwszego nagłówka Markdown w pierwszych 40 liniach.
2. Próbuje znaleźć datę w nazwie pliku albo początku tekstu.
3. Rozpoznaje typ dokumentu po słowach typu `zarządzenie`, `regulamin`, `instrukcja`, `aneks`.
4. Sprawdza, czy dokument wygląda na zmianę/aneks/uchylenie.
5. Dzieli tekst na chunki po nagłówkach `##`, `###`, `####` albo gdy fragment przekroczy ok. 2600 znaków i pojawi się pusta linia.

Każdy chunk ma m.in.:

- `id`,
- `path`,
- `title`,
- `date`,
- `heading`,
- `start_line`,
- `end_line`,
- `text`,
- `is_change`.

Dzięki temu później można pokazać źródło i konkretne linie w dokumencie.

Krok 2: Nadanie Identyfikatorów Źródeł Po wyszukaniu backend wywołuje:

```
attach_source_ids(...)
```

To funkcja z `llm.py`.

Dodaje do kolejnych fragmentów identyfikatory:

S1, S2, S3, ...

Czyli pierwszy znaleziony fragment dostaje `source_id = "S1"`, drugi S2 itd.

Te identyfikatory są bardzo ważne, bo model ma obowiązek cytować odpowiedź w formacie [S1], [S2].

Krok 3: Budowanie Kontekstu dla Gemini `answer_with_gemini()` jest w `llm.py`.

Jeśli nie ma skonfigurowanego klucza `GOOGLE_API_KEY` / `GEMINI_API_KEY`, Gemini nie jest wywoływane. Aplikacja od razu przechodzi do fallbacku i pokazuje najlepsze znalezione źródła.

Jeśli klucz istnieje, aplikacja buduje kontekst przez `build_context()` w `llm.py`.

Każdy fragment jest zapisywany mniej więcej tak:

```
[S1]
Tytuł: ...
Data: ...
Plik: ...
Sekcja: ...
Linie: ...
Dokument zmieniający/aneks/uchylenie: tak/nie
Treść:
...
```

Fragmenty są oddzielane separatorami:

Kontekst jest dokładany tylko do limitu `MAX_CONTEXT_CHARS`. Domyślnie to 18000 znaków. Jeśli kolejne źródło przekroczyłoby limit, pętla się zatrzymuje.

Prompt do Modelu Następnie powstaje prompt:

```
Pytanie użytkownika:
...
```

```
Fragmenty dokumentów:
...
```

Przygotuj odpowiedź dla pracownika instytutu. Nie wychodź poza podane fragmenty.

Model dostaje też instrukcję systemową z `llm.py`.

Instrukcja mówi m.in.:

- odpowiadaj jako asystent IH PAN,
- odpowiadaj wyłącznie na podstawie przekazanych fragmentów,
- jeśli fragmenty nie wystarczają, powiedz to wprost,
- każde twierdzenie o treści dokumentów cytuj jako [S1], [S2],

- jeśli źródło wygląda na aneks/zmianę/uchylenie, zaznacz to,
- nie udzielaj porady prawnej,
- odpowiadaj po polsku, rzeczowo i zwięźle.

Gemini jest wywoływane tutaj: llm.py.

Parametry:

```
model=GEMINI_MODEL
temperature=0.2
maxOutputTokens=1800
```

Niska temperatura oznacza, że odpowiedzi mają być raczej stabilne i mniej kreatywne.

Fallback Bez Gemini Jeśli nie ma klucza API albo wywołanie Gemini rzuci wyjątek, działa `fallback_answer()` z llm.py.

Fallback:

- jeśli nie znaleziono fragmentów, zwraca komunikat, że nie ma podstaw do odpowiedzi,
- jeśli są fragmenty, pokazuje do 5 najlepszych źródeł z krótkim snippetem.

Wtedy nie ma właściwej odpowiedzi modelu, tylko lista pasujących fragmentów.

Krok 4: Wybranie Źródeł do Pokazania Po uzyskaniu odpowiedzi backend wywołuje:

```
sources = cited_sources(answer, chunks)
```

To funkcja z llm.py.

Regex szuka w odpowiedzi wystąpień typu:

```
S1
S2
S3
```

Nie wymaga nawiasów, więc złapie zarówno `[S1]`, jak i samo `S1`.

Potem backend zwraca tylko te chunki, które faktycznie zostały zacytowane przez model.

Jeśli źródeł nie znaleziono, ale odpowiedź jest fallbackiem o braku klucza albo błędzie Gemini, backend pokazuje pierwsze 5 chunków:

```
if not sources and ("Nie skonfigurowano klucza" in answer or "Nie udało się połączyć z Gemini")
    sources = chunks[:5]
```

Na końcu `/api/chat` zwraca JSON:

```
{
  "answer": "...",
  "sources": [...],
```

```

    "stats": {
      "documents": ...,
      "chunks": ...
    }
  }
}

```

Krok 5: Wyświetlenie Odpowiedzi w Chacie Frontend odbiera odpowiedź w `static/app.js`.

Tymczasowa wiadomość Szukam w dokumentach... zostaje zastąpiona treścią odpowiedzi:

```

setMessageContent(
  pending.querySelector(".message-content"),
  data.answer || data.error || "Brak odpowiedzi.",
  true
);

```

Ponieważ trzeci argument to `true`, odpowiedź jest renderowana jako prosty Markdown.

Renderer jest lokalny, własny, w `static/app.js`. Obsługuje:

- nagłówki Markdown,
- listy punktowane,
- listy numerowane,
- inline code,
- pogrubienie,
- kursywę.

Przed renderowaniem inline tekst jest escapowany przez `escapeHtml()`, więc model nie powinien móc łatwo wstrzyknąć HTML-a do odpowiedzi.

Potem frontend dodaje przycisk kopiowania:

```
addCopyButton(pending)
```

i renderuje źródła:

```
renderSources(pending, data.sources || [])
```

Wyświetlanie Źródeł Źródła są renderowane w `renderSources()` w `static/app.js`.

Pod odpowiedzią powstaje element `<details>` z napisem:

Źródła (N)

Każde źródło pokazuje:

- identyfikator [S1],
- tytuł dokumentu,
- datę,
- nagłówek/sekcję,

- zakres linii,
- przycisk ze ścieżką pliku.

Przycisk źródła ma atrybuty:

```
data-document-path="..."
data-start-line="..."
data-end-line="..."
```

Dzięki temu kliknięcie źródła może otworzyć dokument dokładnie w miejscu, z którego pochodził fragment.

Podgląd Dokumentu Źródłowego Kliknięcia w źródła obsługuje listener w `static/app.js`.

Jeśli użytkownik kliknie źródło, wywoływane jest:

```
openDocument(path, startLine, endLine)
```

Ta funkcja robi request:

```
GET /api/document?path=...
```

Backend obsługuje go w `app.py`.

Ważne zabezpieczenie: backend bierze tylko nazwę pliku przez `Path(requested_path).name`, dokleja ją do `MD_DIR`, robi `.resolve()` i sprawdza, czy finalna ścieżka nadal jest bezpośrednio w katalogu `md/` oraz czy ma rozszerzenie `.md`.

Jeśli wszystko jest OK, zwraca:

```
{
  "path": "md/nazwa.md",
  "title": "...",
  "content": "pełna treść markdown"
}
```

Frontend renderuje dokument w modalu przez `showModal()` i `renderDocumentMarkdown()` w `static/app.js`. Linie odpowiadające źródłu dostają klasę `source-hit`, a pierwszy trafiony element jest przewijany do środka widoku.

Enter i Shift+Enter Textarea ma osobną obsługę klawiatury w `static/app.js`.

- **Enter** wysyła formularz.
- **Shift+Enter** pozwala wpisać nową linię.
- Puste pytanie nie jest wysyłane.

Najważniejsza Sekwencja w Skrócie 1. Użytkownik wpisuje pytanie w textarea. 2. `static/app.js` przechwytuje submit formularza. 3. Wiadomość użytkownika trafia do DOM. 4. Pojawia się tymczasowa odpowiedź: Szukam w dokumentach.... 5. Frontend wysyła `POST /api/chat` z `{ message }`. 6. Flask sprawdza sesję logowania. 7. `/api/chat` czyści i waliduje pytanie. 8. `kb.search()` tokenizuje pytanie i znajduje pasujące fragmenty Markdown. 9. `attach_source_ids()` dodaje źródła `S1, S2, ...` 10. `answer_with_gemini()`

buduje prompt z pytaniem i fragmentami. 11. Gemini generuje odpowiedź wyłącznie na podstawie przekazanego kontekstu. 12. `cited_sources()` wykrywa, które źródła model zacytował. 13. Backend zwraca JSON z odpowiedzią i źródłami. 14. Frontend podmienia placeholder na odpowiedź. 15. Odpowiedź jest renderowana jako prosty Markdown. 16. Pod odpowiedzią pojawia się rozwijana lista źródeł. 17. Kliknięcie źródła otwiera pełny dokument w modalu i podświetla odpowiednie linie.

Istotne Ograniczenia Aplikacja robi wyszukiwanie leksykalne, nie semantyczne. Jeśli użytkownik zada pytanie innymi słowami niż występują w dokumentach, dobór źródeł może być słabszy.

Model nie widzi całego katalogu dokumentów, tylko fragmenty wybrane przez `kb.search()`. Jeśli wyszukiwarka nie wybierze właściwego fragmentu, Gemini nie ma jak poprawnie odpowiedzieć.

Źródła pokazywane pod odpowiedzią zależą od tego, czy model użył cytowań S1, S2 itd. Instrukcja systemowa tego wymaga, ale kod nie waliduje twardo, czy każde twierdzenie faktycznie ma cytowanie.

Aplikacja ładuje indeks przy starcie. Jeśli ktoś zmieni pliki w `md/` podczas działania serwera, globalne `kb` samo się nie przeładuje, chyba że aplikacja zostanie zrestartowana albo kod zostanie rozszerzony o ręczne `kb.reload()`.